

Objetivos:

- Comprender cómo una arquitectura de servicios responde ante distintos tipos de tráfico.
- Relacionar componentes de infraestructura cloud con conceptos de redes: ruteo, balanceo de carga, almacenamiento, bases de datos, caché, colas y filtrado de tráfico malicioso.
- Analizar fallas, cuellos de botella y decisiones de escalabilidad.

Requisitos:

- Computadora(s) con acceso a internet.
- Navegador web moderno.

Contexto

En este laboratorio vamos a usar Server Survival como simulador de infraestructura. El juego representa un sistema que recibe tráfico de distintos tipos y que debe procesarlo correctamente manteniendo presupuesto, reputación y salud de los servicios.

Aunque el entorno es lúdico, los conceptos son reales: un sistema mal diseñado puede fallar por sobrecarga, mala distribución del tráfico, ausencia de caché, falta de filtrado ante ataques o una base de datos saturada.

Despliegue del juego

Podemos clonar el repositorio <https://github.com/pshenok/server-survival> o utilizar el repo desplegado <https://pshenok.github.io/server-survival/> (aunque esta opción podría no estar disponible). Abran index.html en el navegador y prueben!

1) Reconocimiento de arquitectura

Ingresa al juego y observa los componentes disponibles.

Identificá qué función cumple cada uno de estos elementos:

● Firewall ● Load Balancer ● Queue ● Compute ● Serverless Function ● SQL DB	● NoSQL ● Cache ● CDN ● Storage ● Search Engine ● Réplica
---	---

Para cada uno, respondé brevemente:

- ¿Qué problema resuelve?
- ¿En qué capa o capas del modelo TCP/IP podríamos ubicar su función principal?
- ¿Qué pasaría si ese componente falta en una arquitectura real?

2) Tipos de tráfico

El simulador trabaja con distintos tipos de solicitudes: STATIC, READ, WRITE, UPLOAD, SEARCH, MALICIOUS

Para cada tipo de tráfico, completar una tabla con:

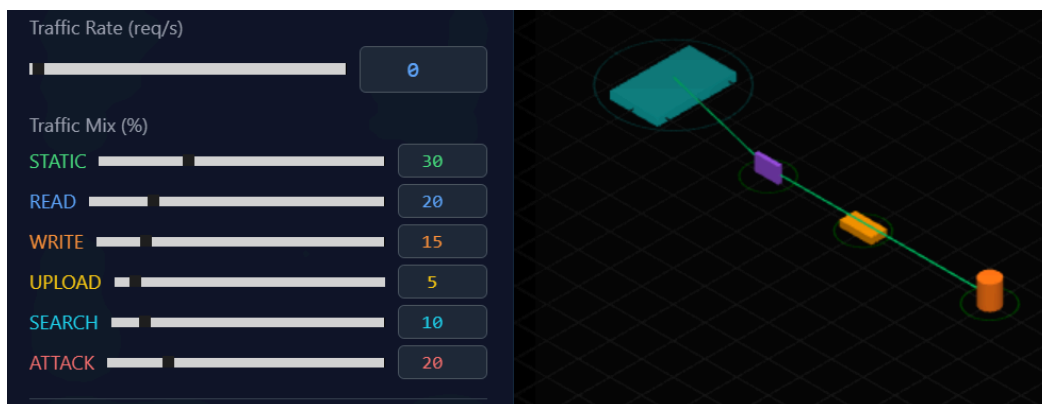
- Tipo de tráfico
- Ejemplo real
- Componente recomendado para procesarlo
- Riesgo si se procesa incorrectamente

Ejemplo:

STATIC puede representar imágenes, CSS o JavaScript de una página web. En una arquitectura real podría ser servido por un CDN o almacenamiento estático. Si lo procesamos desde servidores de aplicación, podemos desperdiciar capacidad de cómputo.

3) Testeamos queues

Construiremos una infraestructura mínima para testear queues. Desplieguen el siguiente esquema en modo Sandbox:



Conectarán un firewall, una queue y una instancia de computación. Denle play! y jueguen con el throughput (rate de tráfico). Incrementen el rate: que sucede después de la queue?. Mantengan el rate alto y luego llevenlo a cero rápidamente. Qué sucede después de la queue?

4) Primera infraestructura mínima

Tu arquitectura debe intentar resolver:

- Tráfico estático y uploads.
- Lecturas y escrituras de datos.
- Búsquedas.
- Ataques o tráfico malicioso.

En modo sandbox, modificá la distribución de tráfico y modificá el rate. Documenta con capturas:

- a) La arquitectura inicial.
- b) El presupuesto inicial.
- c) El estado de salud de los servicios.
- d) El momento en que la arquitectura empieza a fallar, si ocurre.

Respondé:

¿Qué componente falló primero?

¿Por qué creés que falló?

¿Fue un problema de capacidad, diseño, costo o seguridad?

5) Escalabilidad y balanceo

Modificá la arquitectura del punto anterior para soportar mayor tráfico.

Probá al menos dos estrategias distintas:

- Agregar más capacidad de cómputo.
- Agregar balanceador de carga.
- Agregar caché.
- Agregar réplicas de lectura.
- Agregar cola de mensajes.
- Separar servicios según tipo de tráfico.

Para cada estrategia, documentá:

¿Escalar horizontalmente siempre mejora el sistema? Justificá usando evidencia del simulador.

6) Sobrevivir

Diseñá una arquitectura inicial sólida y tratá de sobrevivir lo más posible mejorando la misma en el modo “survival”. Documenta tu arquitectura final (al momento del fallo) con una captura y explicá:

- Por qué elegiste cada componente.
- Qué tráfico atiende cada uno.
- Qué cuello de botella apareció primero..
- Qué componente escalarías si tuvieras más presupuesto.

Si logras “estabilidad”, en el sentido que ganas más dinero que el costo de expandir la arquitectura, podés considerar que “ganaste”. Más de 300 mil puntos puede llegar a suponer una condición así.

El récord de los profes:

